目录

源代码与使用说明 **1**
1. 项目说明 . . . 1
2. 目录结构 . . . 1
3. 运行环境 . . . 2
4. 运行前检查 . . . 2
5. 上传脚本 . . . 2
6. 数据清洗 . . . 2
7. V1：分组 IQR Baseline . . . 3
    7.1 运行逻辑 . . . 3
    7.2 运行命令 . . . 3
8. V2：分组 IQR + Z-score . . . 3
    8.1 运行逻辑 . . . 3
    8.2 先运行小数据类型 . . . 3
    8.3 同一会话顺序运行其余类型 . . . 3
9. V3：Window Z-score . . . 4
    9.1 运行逻辑 . . . 4
    9.2 运行命令 . . . 4
10. V4：MultiMetric Fusion . . . 4
    10.1 运行逻辑 . . . 4
    10.2 运行命令 . . . 4
11. 结果检查 . . . 5
12. 推荐运行顺序 . . . 5

源代码与使用说明

1. 项目说明

本项目使用 HDFS 存储 AIOps 监控数据，使用 Spark DataFrame/SQL 完成数据清洗和四个递进式时间序列异常检测方案：

```
V1：分组 IQR Baseline
V2：分组 IQR + Z-score
V3：Window Z-score
V4：MultiMetric Fusion
```

2. 目录结构

```
源代码与使用说明/
├── 使用说明.md
├── data_clean/
│   └── spark_read_clean.scala          # Spark 清洗脚本
├── V1_IQR/
│   └── iqr_by_type_fast.scala
├── V2_IQR_Zscore/
│   ├── iqr_zscore_by_type_fast.scala
│   └── iqr_zscore_all_types_fast.scala
├── V3_Window_Zscore/
│   └── window_zscore_by_type_fast.scala
└── V4_MultiMetric_Fusion/
    └── multimetric_fusion_by_type_fast.scala
```

`iqr_zscore_by_type_fast.scala` 每次处理一类指标；`iqr_zscore_all_types_fast.scala` 在同一 Spark 会话中顺序处理多类指标，用于减少重复启动 Spark Shell 的时间。

3. 运行环境

组件	实验版本
操作系统	CentOS 7
Java	OpenJDK 8
Hadoop	3.3.6
Spark	3.5.1
Scala	2.12

实验使用 1 个 Master 和 2 个 Worker。算法脚本以 Spark Standalone 方式提交：

```
spark://master:7077
```

HDFS NameNode 地址：

```
hdfs://master:9000
```

4. 运行前检查

所有命令在 Master 节点执行。

```
jps
hdfs dfsadmin -report
hdfs dfs -ls /aiops/metric
```

确认 Spark Master 和 Worker 已启动，HDFS 中已有原始数据。检测脚本运行前还需确认清洗结果存在：

```
hdfs dfs -ls /aiops/output/clean_metrics
```

5. 上传脚本

将本目录中的 Scala 文件上传到 Master 节点的 `/root` 目录，并保持下列文件名：

```
/root/spark_read_clean.scala
/root/iqr_by_type_fast.scala
/root/iqr_zscore_by_type_fast.scala
/root/iqr_zscore_all_types_fast.scala
/root/window_zscore_by_type_fast.scala
/root/multimetric_fusion_by_type_fast.scala
```

6. 数据清洗

原始数据路径：

```
hdfs://master:9000/aiops/metric
```

运行命令：

```
spark-shell --master local[*] -i /root/spark_read_clean.scala
```

清洗内容包括类型转换、关键字段空值过滤、`service` 宽表转长表和字段统一。输出为 Parquet：

```
hdfs://master:9000/aiops/output/clean_metrics
```

统一核心字段：

```
timestamp, object_id, kpi_name, metric_type, value
```

7. V1：分组 IQR Baseline

7.1 运行逻辑

按 `object_id + kpi_name + metric_type` 分组计算 Q1、Q3、IQR 和上下界，再将统计量 join 回数据点判断异常。

7.2 运行命令

将 `METRIC_TYPE` 依次设置为 `service`、`node`、`jvm`、`istio`、`container`：

```
METRIC_TYPE=service spark-shell \
--master spark://master:7077 \
--driver-memory 3g \
--executor-memory 2g \
--total-executor-cores 4 \
--conf spark.sql.shuffle.partitions=16 \
--conf spark.network.timeout=600s \
--conf spark.executor.heartbeatInterval=60s \
-i /root/iqr_by_type_fast.scala
```

数据量较大时，建议 `jvm/istio` 使用 32 个 shuffle 分区，`container` 使用 64 个 shuffle 分区和 4GB driver 内存。

输出：

```
/aiops/output/iqr_baseline_fast/iqr_stats/<metric_type>
/aiops/output/iqr_baseline_fast/anomaly_points/<metric_type>
```

8. V2：分组 IQR + Z-score

8.1 运行逻辑

在 V1 的分组统计上增加均值、标准差和 Z-score，默认使用严格规则：

```
IQR 异常 AND |z_score| > 3.0
```

8.2 先运行小数据类型

```
METRIC_TYPE=service FINAL_RULE=and Z_THRESHOLD=3.0 WRITE_SUMMARY=false spark-shell \
--master spark://master:7077 \
--driver-memory 3g \
--executor-memory 2g \
--total-executor-cores 4 \
--conf spark.sql.shuffle.partitions=16 \
--conf spark.network.timeout=600s \
--conf spark.executor.heartbeatInterval=60s \
-i /root/iqr_zscore_by_type_fast.scala
```

8.3 同一会话顺序运行其余类型

```
METRIC_TYPES=node,jvm,istio,container FINAL_RULE=and Z_THRESHOLD=3.0 WRITE_SUMMARY=false spark-she
--master spark://master:7077 \
--driver-memory 4g \
```

```
--executor-memory 2g \
--total-executor-cores 4 \
--conf spark.sql.shuffle.partitions=64 \
--conf spark.network.timeout=600s \
--conf spark.executor.heartbeatInterval=60s \
-i /root/iqr_zscore_all_types_fast.scala
```

输出：

```
/aiops/output/iqr_zscore_fast/stats/<metric_type>
/aiops/output/iqr_zscore_fast/anomaly_points/<metric_type>
```

9. V3：Window Z-score

9.1 运行逻辑

在每条序列内按时间排序，用当前点和前 100 个历史点计算局部均值、标准差和 Z-score。默认要求窗口至少有 30 个点。

9.2 运行命令

```
METRIC_TYPE=service Z_THRESHOLD=3.0 WINDOW_ROWS=100 MIN_WINDOW_POINTS=30 spark-shell \
--master spark://master:7077 \
--driver-memory 3g \
--executor-memory 2g \
--total-executor-cores 4 \
--conf spark.sql.shuffle.partitions=16 \
--conf spark.network.timeout=600s \
--conf spark.executor.heartbeatInterval=60s \
-i /root/window_zscore_by_type_fast.scala
```

按 V1 的顺序修改 `METRIC_TYPE` 处理其他类型，并对 `jvm/istio/container` 相应使用 32/32/64 个 shuffle 分区。

输出：

```
/aiops/output/window_zscore_fast/anomaly_points/<metric_type>
/aiops/output/window_zscore_fast/series_stats/<metric_type>
```

10. V4：MultiMetric Fusion

10.1 运行逻辑

选择每类出现频率最高的 Top K KPI，按 `timestamp + object_id + metric_type` 做 pivot，将多个 KPI 放到同一行。当同一行至少有 2 个 KPI 满足 `|z| > 3.0` 时，判为多指标融合异常事件。

10.2 运行命令

```
METRIC_TYPE=service TOP_K=30 Z_THRESHOLD=3.0 MIN_ANOMALY_METRICS=2 spark-shell \
--master spark://master:7077 \
--driver-memory 3g \
--executor-memory 2g \
--total-executor-cores 4 \
--conf spark.sql.shuffle.partitions=16 \
--conf spark.network.timeout=600s \
--conf spark.executor.heartbeatInterval=60s \
-i /root/multimetric_fusion_by_type_fast.scala
```

修改 METRIC_TYPE 依次处理其他类型。TOP_K 越大，pivot 生成的宽表列数越多，运行时间和内存开销也越大。

输出：

```
/aiops/output/multimetric_fusion_fast/anomaly_points/<metric_type>
/aiops/output/multimetric_fusion_fast/summary/<metric_type>
/aiops/output/multimetric_fusion_fast/selected_kpis/<metric_type>
```

11. 结果检查

每类运行完成后，先检查输出目录中是否存在 _SUCCESS。例如：

```
hdfs dfs -ls /aiops/output/iqr_baseline_fast/anomaly_points/service
hdfs dfs -ls /aiops/output/iqr_zscore_fast/anomaly_points/service
hdfs dfs -ls /aiops/output/window_zscore_fast/anomaly_points/service
hdfs dfs -ls /aiops/output/multimetric_fusion_fast/anomaly_points/service
```

统计建议在检测任务完成后单独进行，不要在主脚本中反复执行 count()、show() 和 orderBy()，以免不必要的重复计算。

进入 Spark Shell：

```
spark-shell --master spark://master:7077
```

读取某一类异常结果：

```
val anomalies = spark.read.parquet(
  "hdfs://master:9000/aiops/output/iqr_zscore_fast/anomaly_points/service"
)

anomalies.count()

anomalies
  .groupBy("object_id", "kpi_name", "metric_type")
  .count()
  .orderBy(desc("count"))
  .show(20, false)
```

12. 推荐运行顺序

1. 运行数据清洗，生成 clean_metrics
2. 先用 service 检查每个脚本和输出路径
3. 再按 node → jvm → istio → container 顺序运行
4. 每类结果写入 HDFS 后再单独统计

container 和 istio 数据量最大，建议最后运行。任务异常缓慢时，应检查 Spark UI、Worker 状态、HDFS 状态及是否有残留的 SparkSubmit 进程。